学习与优化

学习笔记

◎ Editors 夏同

Beautybook

前言

本书所讲解的的知识内容来自笔者在 B 站观看的课程《学习与优化》，讲课老师为华南理工大学龚老师。

— Ethan Lu

2026 年 3 月 19 日

目录

第一部分　深度学习 1

第一章　神经网络与深度学习基础 3

1.1 启发性思考 3
1.2 神经元：深度学习的基石 3
1.2.1 为什么需要神经元？ 3
1.2.2 类比人类神经元 3
1.3 神经网络的训练：让模型学会思考 5
1.3.1 第一步：定义损失函数 5
1.3.1.1 回归任务：预测连续值 5
1.3.1.2 分类任务：预测离散类别 5
1.3.2 第二步：梯度下降 7
1.3.3 第三步：计算梯度——反向传播 9
1.3.4 计算权重梯度 10
1.4 常见的激活函数及其特点 12
1.4.1 发行版安装与更新 16
1.4.2 其他发行版本 17

第二章　beautybook 设置说明 18

2.1 语言模式 18
2.2 颜色主题 18
2.3 封面 19
2.3.1 封面个性化 19
2.3.2 封面图 19
2.3.3 徽标 19
2.3.4 自定义封面 19
2.4 章标题 19
2.5 数学环境简介 20
2.5.1 定理类环境的使用 20
2.5.1.1 amsthm 20
2.5.1.2 thmtools 20
2.5.1.3 彩色强调盒子 20
2.5.1.4 个人版权的盒子共两种 21

2.5.2 修改计数器 22
2.5.3 自定义定理类环境 22
2.6 列表环境 24
2.7 参考文献 24
2.7.1 打印文献 24
2.7.2 修改文献格式 24

第三章 字体选项 25

3.1 数学字体选项 25

学习与优化

Part I

深度学习

深度学习，机器学习等知识是该课程的主要内容

1 神经网络与深度学习基础

Part I

Sec 1.1 启发性思考

深度学习的核心思想是模仿人脑处理信息的方式。人脑中约有 860 亿个神经元，每个神经元通过树突接收信号，在细胞体整合，若超过某个阈值则通过轴突释放信号（电脉冲）至其他神经元。这一“全有或全无”的激发模式启发了最早的数学模型——**人工神经元**（或称感知器）。

本章将系统学习：

- 人工神经元的基本组成：**权重**、**偏置**、**激活函数**
- 几种经典激活函数（Sigmoid, tanh, ReLU）的特性与对比
- **前馈神经网络**的结构与矩阵表示
- **Softmax 函数**及其温度系数的妙用
- 神经网络训练的完整流程：**前向传播**、**损失函数**、**梯度下降**与核心算法——**反向传播（Backpropagation, BP）**

Sec 1.2 神经元：深度学习的基石

1.2.1 为什么需要神经元？

人工神经网络的设计灵感源于对生物神经系统的简化模拟。在大脑中，数十亿个神经元通过复杂的连接处理信息：每个神经元接收来自其他神经元的信号，当这些信号的累积超过某个阈值时，该神经元被激活，并向下一级神经元传递自己的信号。模仿这一机制，我们可以构造一个简单的数学单元（人工神经元）作为深度学习模型的基本构件。它的目标是从输入数据中学习出有意义的特征表示，从而完成分类、回归等任务。

1.2.2 类比人类神经元

因此，为了构造人工神经元的数学模型，我们需要思考人类的行为，再进行类比。如果把神经元比作一个人在听取多方意见后做决定，那么权重就类似于你对每个朋友发言的“信任度”或“重视程度”。有些朋友（高权重）的话会极大地影响你的最终决定，而另一些朋友（低权重）则可有可无。权重可以是正的（朋友支持）也可以是负的（朋友反对），甚至为零（忽略）。

从数学上看，权重和输入的关系其实是一个简单的“加权求和”过程：我们把每个输入 x_i 乘以对应的权重 w_i，再把所有乘积加起来，最后加上偏置 b。写成公式就是 $z = w_1x_1 + w_2x_2 +$

$\cdots + w_n x_n + b$。这个公式用向量符号可以简洁地表示为 $z = \mathbf{w} \cdot \mathbf{x} + b$，其中 $\mathbf{w} \cdot \mathbf{x}$ 表示两个向量对应分量相乘再相加，即“点积”。点积的几何意义可以理解为：向量 $\mathbf{x}$ 在向量 $\mathbf{w}$ 方向上的投影长度再乘以 $\mathbf{w}$ 的长度。换句话说，权重向量 $\mathbf{w}$ 指定了一个“方向”，而加权和 z 的大小反映了输入 $\mathbf{x}$ 在这个方向上的“分量”。

有人会问，公式 $z = \mathbf{w} \cdot \mathbf{x} + \mathbf{b}$ 中，后面的 $\mathbf{b}$ 是什么呢？观察公式，我们会发现，即使没有任何外部信息（即所有输入 $x_i = 0$），神经元也可能有被激活的倾向，这就是偏置 b 的作用。延续上面的类比，偏置可以看作你与生俱来的性格底色，或者在没有外界意见时的“预设立场”。如果偏置为正，那么即便没有输入信号，神经元也倾向于兴奋；反之，如果偏置为负，则需要较强的外部信号才能将其激活。从几何上讲，偏置使得决策边界不必通过原点，提供了平移的自由度，从而显著增强模型的表达能力。例如在二维平面中，若没有偏置，分类直线必须穿过原点，这显然过于局限；偏置允许我们将直线移动到任何位置。

那么权重和偏置如何决定一个分类决策呢？想象一个二维平面，每个输入点 (x_1, x_2) 要么属于类别 A，要么属于类别 B。一个神经元可以学会用一条直线来划分这两个类别——这条直线就是所谓的“决策边界”（在高维空间中，它被称为“超平面”，其实就是直线、平面概念的推广）。权重向量 $\mathbf{w}$ 的方向正好垂直于这条直线，决定了直线的倾斜角度；而偏置 b 决定了直线离开原点的距离。改变权重，就相当于旋转这条直线；改变偏置，就相当于平移它。训练神经网络的过程，就是通过不断微调这些权重和偏置，让这条直线（或超平面）能够恰好将不同类别的点分开，使得神经元的输出 $a = f(z)$ 尽可能地接近我们期望的标签（比如类别 A 输出 1，类别 B 输出 0）。这种调整靠的是从大量例子中“学习”：给网络看很多带标签的样本，让它自己摸索出最合适的直线位置。

然而，神经元最终是要组成神经网络的。倘若只有加权和与偏置，那无论堆叠多少层神经元，整个网络仍然等价于一个线性模型（因为线性函数的复合仍是线性的），永远无法模拟现实世界中千变万化的非线性现象。激活函数的引入打破了这一僵局，它给神经元注入了“灵魂”。

Definition 1.2.1. (神经元的数学模型)

一个典型的人工神经元接收 n 个输入 $x_1, x_2, \ldots, x_n$（例如，一张图片的像素值或一条文本的词向量）。每个输入都对应一个可学习的参数，即“权重” w_i，它决定了该输入对神经元输出的重要性。此外，神经元还有一个额外的参数“偏置” b，它可以理解为神经元的“固有倾向”。首先，神经元计算输入的加权和（称为“预激活值”）：

$$z = \sum_{i=1}^{n} w_i x_i + b$$

再将 z 送入一个非线性函数 f（称为“激活函数”），得到神经元的最终输出：

$$a = f(z)$$

上文提到，如果没有非线性，无论神经元堆叠多少层，整个神经网络仍然等价于一个线性模型，无法学习复杂的模式。这是为什么呢？让我们从一个直观例子入手：假设我们想用一条直线将平面上的两类点分开。如果这两类点是线性可分的（比如被一条直线分开），那么一个简单的线性模型就足够了。但现实中的数据往往更复杂，比如经典的“异或”（XOR）问题：两个输入 $(0,0)$ 和 $(1,1)$ 属于一类，$(0,1)$ 和 $(1,0)$ 属于另一类。你无法用任何一条直线将它们分开，必须使用曲线或折线。线性模型（无论多少层线性变换的叠加）永远无法画出这

样的曲线——因为线性变换的组合仍然是线性的。激活函数提供的正是这种“弯曲空间”的能力：它让神经元能够根据输入的不同，选择性地“激活”或“抑制”，从而在几何上对决策边界进行扭曲，使得原本纠缠的数据得以分离。

从数学上讲，激活函数使神经网络成为**通用函数逼近器**：只要网络足够宽（每层神经元足够多）或足够深（层数足够多），它就能以任意精度逼近任意复杂的连续函数。

Sec 1.3 神经网络的训练：让模型学会思考

在深入具体的激活函数之前，我们先思考一个问题：什么样的函数适合作为神经元的激活函数？上文提到，正是因为神经网络需要非线性的因子，才需要我们找到激活函数，所以在学习并寻找激活函数之前，我们要先了解神经网络的构建。

我们已经知道，神经网络通过调整权重和偏置来完成任务。但如何调整呢？首先需要量化网络表现的好坏。换句话说，训练一个神经网络，本质上就是找到一个最优的参数集合（所有权重和偏置），使得模型在给定数据上的表现尽可能好。这个过程可以被抽象为一个数学优化问题：我们定义一个函数来衡量模型当前的表现有多糟糕，然后不断地调整参数，让这个函数的值越来越小。

1.3.1 第一步：定义损失函数

Remark. “猜对了，有奖励；猜错了，有惩罚。”

为了量化模型预测的好坏，我们引入**损失函数**（Loss Function）的概念。损失函数 L 接受模型的预测值 $\hat{y}$ 和真实标签 y，输出一个非负的数值，数值越小表示预测越准确。换句话说，损失函数就像一位严厉的裁判，给模型的表现打分，我们的目标就是让这个分数尽可能低。

然而，不同类型的问题需要不同的“裁判规则”。这就好比射击比赛和体操比赛的评分标准完全不同——射击看环数（数值差距），体操看动作规范（类别匹配）。在机器学习中，我们主要面对两类任务，它们各自有最常用的损失函数。

1.3.1.1 回归任务：预测连续值

这类问题的目标是预测一个具体的数值，比如房价、温度、股票价格。模型输出的是一个实数，我们需要衡量它和真实值之间的差距。最直观的方式就是计算它们的**差的平方**——差得越多，惩罚越大。这就是**均方误差（Mean Squared Error, MSE）**。假设有 m 个样本，第 i 个样本的真实值为 $y^{(i)}$，模型预测值为 $\hat{y}^{(i)}$，则

$$J(W,b)=\frac{1}{2m}\sum_{i=1}^{m}(\hat{y}^{(i)}-y^{(i)})^2.$$

这里的 $\frac{1}{2}$ 是为了后续求导时与平方项的系数 2 抵消，使公式更简洁，不影响优化结果（因为乘以一个常数不会改变最小值的位置）。

1.3.1.2 分类任务：预测离散类别

这类问题的目标是判断一个样本属于哪个类别，比如识别图片中是猫还是狗，或者手写数字是 0-9 中的哪一个。模型输出的通常是一个概率分布，通俗来讲，就是“你有几成把握”（例

如，70% 是猫，30% 是狗，0% 是鸟）。此时，平方误差就不太合适了——它无法很好地衡量两个概率分布之间的差异。

因此，在分类任务中，真实标签（即每个样本的正确类别）需要以另一种计算机能理解的方式表示。最直观的想法是用数字直接表示类别：比如用“1”代表猫，“2”代表狗，“3”代表鸟。但这种表示方式隐含了一个问题：数字之间的大小关系会被模型误解——难道猫和狗的距离比猫和鸟更近吗？显然不是。类别之间是平等的离散关系，没有数值上的大小顺序。我们可以引入数学中的“示性函数”，定义计算机的 one-hot 编码形式：

Definition 1.3.2. (*one-hot* 编码)

在分类任务中，真实标签通常需要转换为一种便于模型处理的数值形式。假设共有 K 个类别，并约定一个固定的顺序（例如猫、狗、鸟）。对于某个样本，若它属于第 c 类，则其 one-hot 编码是一个长度为 K 的向量，其中第 c 个分量为 1，其余分量均为 0。形式化地：

$$y_k = \begin{cases} 1, & k = c, \\ 0, & k \neq c, \end{cases} \qquad k = 1, 2, \ldots, K.$$

这种表示方式有两个重要性质：

- 它消除了类别之间的假想顺序，使每个类别在向量空间中处于平等地位；
- 它可以看作一个“退化的概率分布”——将全部概率质量 1 赋予正确类别，其他类别概率为 0。

因此，在交叉熵损失等涉及概率度量的函数中，one-hot 编码能够自然地与模型输出的预测概率进行比较。

有了 one-hot 编码，真实标签被表示为一个“退化”的概率分布——在正确类别上概率为 1，其余为 0。现在，模型输出的是预测概率分布 $\hat{y}_k$（通常由 Softmax 层产生）。我们的目标是让预测分布尽可能接近真实分布。但如何量化两个分布之间的“接近程度”呢？

让我们从一个直观的想法出发：对于每个样本，我们希望模型给正确类别的预测概率 $\hat{y}_c$ 尽可能大。因为如果模型总是把高概率赋给正确类别，说明它分类很准。对于整个数据集，一个自然的想法是把所有样本正确类别的预测概率乘起来，得到一个总的“信心值”：

$$\text{总信心} = \prod_{i=1}^{m} \hat{y}_{c_i}^{(i)}$$

这个乘积越大，说明模型整体上越自信且越正确。因此，我们可以把最大化这个乘积作为训练目标。然而，直接最大化乘积在数学上不太方便（连乘容易导致数值下溢，且求导复杂）。但我们可以利用一个技巧：由于对数函数是单调递增的，最大化乘积等价于最大化它的对数。于是：

$$\log\left(\prod_{i=1}^{m} \hat{y}_{c_i}^{(i)}\right) = \sum_{i=1}^{m} \log \hat{y}_{c_i}^{(i)}$$

现在变成了求和，容易处理。但我们通常习惯最小化损失函数，所以取负号，并除以样本数 m 得到平均损失：

$$J = -\frac{1}{m} \sum_{i=1}^{m} \log \hat{y}_{c_i}^{(i)}$$

这就是“负对数似然损失”（Negative Log-Likelihood, NLL）。而由于真实标签是 one-hot 形式，这个表达式正好等于交叉熵损失：

$$J = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log \hat{y}_k^{(i)}$$

因为 $y_k^{(i)}$ 只在 $k = c_i$ 时为 1，其他为 0，所以双重求和退化为上面的单重求和。从这个推导可以看出，交叉熵损失的本质就是“鼓励模型给正确类别赋予高概率”。当 $\hat{y}_c$ 接近 1 时，$\log \hat{y}_c$ 接近 0，损失很小；当 $\hat{y}_c$ 远小于 1 时，$\log \hat{y}_c$ 是绝对值很大的负数，取负后变成很大的正数，从而对模型施加严厉惩罚。这正是分类任务所需的损失函数。

明确了损失函数之后，我们的目标就变得非常清晰：寻找使损失函数 $J(W, b)$ 最小的参数 W 和 b。接下来的问题就是，如何高效地找到这些最优参数？这正是我们下一节要讨论的核心——梯度下降与反向传播。

1.3.2 第二步：梯度下降

如何最小化损失函数？最经典的算法是**梯度下降**（Gradient Descent）。它的思想非常直观：想象你站在一座山上，周围大雾弥漫，看不见整座山的地形。你只能感受到脚下地面的倾斜程度。为了尽快下山，你会朝着最陡的向下方向迈出一小步，然后在新位置重复这个过程。最终，你就能到达山谷的底部。

有了直观的理解之后，我们现在尝试从数学上理解梯度下降，先回顾一下多元微积分中的几个基本概念：假设损失函数 J 是一个关于多个参数（比如所有权重 $w_1, w_2, \ldots$ 和偏置）的多元函数。在某一点 $\mathbf{p}$ 处，函数值的变化趋势由它的偏导数描述：$\dfrac{\partial J}{\partial w_i}$ 表示当其他参数固定时，稍微改变 w_i 会引起 J 变化多少。把所有偏导数组合成一个向量，就得到了该点的“梯度”，记作 $\nabla J(\mathbf{p})$。这个向量的方向指向函数值增长最快的方向，其长度表示增长率的快慢。

Definition 1.3.3. (梯度)

设 $J : \mathbb{R}^n \to \mathbb{R}$ 是一个多元函数，且在点 $\mathbf{p} = (p_1, p_2, \ldots, p_n) \in \mathbb{R}^n$ 的某个邻域内所有一阶偏导数存在。则称向量

$$\nabla J(\mathbf{p}) = \left(\frac{\partial J}{\partial p_1}(\mathbf{p}), \frac{\partial J}{\partial p_2}(\mathbf{p}), \ldots, \frac{\partial J}{\partial p_n}(\mathbf{p})\right)$$

为函数 J 在点 $\mathbf{p}$ 处的**梯度**。梯度的方向是函数在该点处方向导数最大的方向（即函数值增长最快的方向），其模长等于该方向的方向导数。几何上，梯度向量垂直于函数过该点的等值面（或等值线），并指向函数值增加的方向。

现在我们要最小化 J，自然会想到朝着函数值下降的方向移动。从 $\mathbf{p}$ 点出发，沿着负梯度方向 $-\nabla J(\mathbf{p})$ 移动一小步，就能最有效地降低 J。为什么呢？根据泰勒展开的一阶近似：

$$J(\mathbf{p} + \Delta\mathbf{p}) \approx J(\mathbf{p}) + \nabla J(\mathbf{p}) \cdot \Delta\mathbf{p}$$

如果我们令 $\Delta\mathbf{p} = -\eta\nabla J(\mathbf{p})$，其中 $\eta > 0$ 是一个小正数，那么：

$$J(\mathbf{p} - \eta\nabla J) \approx J(\mathbf{p}) - \eta\|\nabla J\|^2 \leqslant J(\mathbf{p})$$

因为 $\|\nabla J\|^2$ 是非负的。所以沿着负梯度方向移动，可以保证（在步长足够小的前提下）函数值下降。这个 η 就是学习率（learning rate），它控制着移动的步长。如果 η 太小，下降会很慢；如果 η 太大，泰勒近似可能失效，甚至导致函数值上升。

Definition 1.3.4. (学习率（*Learning Rate*）)

在梯度下降及其变种算法中，**学习率** $\eta > 0$ 用于控制参数更新的步长。它决定了当前迭代中沿负梯度方向移动的幅度。设 J 为损失函数，$\nabla J(\theta)$ 为当前参数 θ 处的梯度，则参数更新规则为：

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta\nabla J(\theta_{\text{old}})$$

学习率的选择至关重要：若 η 过小，收敛速度缓慢；若 η 过大，参数更新可能越过极小点，导致震荡甚至发散。实践中常采用自适应学习率方法（如 AdaGrad、Adam）或在训练过程中调整学习率（如学习率衰减）来提升优化效果。

将上述思想应用到神经网络的所有参数上，就得到了梯度下降的更新规则：

$$W^{(l)} := W^{(l)} - \eta\frac{\partial J}{\partial W^{(l)}}, \qquad b^{(l)} := b^{(l)} - \eta\frac{\partial J}{\partial b^{(l)}}$$

这里 $\dfrac{\partial J}{\partial W^{(l)}}$ 表示对第 l 层所有权重矩阵的偏导数（实际是一个矩阵，每个元素对应一个权重的偏导），$\dfrac{\partial J}{\partial b^{(l)}}$ 同理。整个过程就是不断重复：计算梯度，沿负方向更新，直到收敛到某个极小值点（可能是局部最小，也可能是全局最小）。

需要注意的是，实际训练中我们通常使用“小批量梯度下降”（mini-batch gradient descent），即每次从训练集中随机抽取一小批样本计算梯度，而不是用全部样本。这样能引入随机性，有助于逃离局部极小点。还可以利用向量化加速：

Definition 1.3.5. (向量化加速)

在深度学习的实现中，我们通常需要同时处理多个样本（例如一个小批量中的 m 个样本）。如果采用逐样本循环的方式依次计算每个样本的梯度，不仅代码效率低下，而且无法充分利用现代硬件（尤其是 GPU）的并行计算能力。而**向量化加速**（Vectorization）是指将循环操作改写为矩阵或张量运算，从而一次性完成所有样本的计算。例如，单个样本的前向传播涉及矩阵乘法 $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$；对于 m 个样本，我们可以将输入 $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}$ 拼接成一个矩阵 X（每列是一个样本），那么所有样本的预激活值可以一步计算为 $Z = \mathbf{W}X + \mathbf{b}$（这里 $\mathbf{b}$ 通过广播机制与每列相加）。这样，原本需要循环 m 次的操作变成了一次矩阵乘法。

因此，小批量梯度下降中的“小批量”不仅提供了梯度估计的随机性，还使得向量化成为可能，从而在保证收敛效果的同时，极大加快了训练速度。

1.3.3 第三步：计算梯度——反向传播

现在的问题变成了：如何高效地计算损失函数 J 对每一层每一个参数的偏导数 $\frac{\partial J}{\partial W^{(l)}}$ 和 $\frac{\partial J}{\partial b^{(l)}}$？神经网络是一个复杂的复合函数，直接求导几乎不可能，但我们可以利用微积分中的**链式法则**，从输出层开始，逐层向后传播误差信号，这就是**反向传播**（Backpropagation）算法。

反向传播并不是一个新的优化方法，它只是一个计算梯度的技巧。一旦我们通过反向传播算出了所有梯度，就可以用梯度下降（或它的各种改进版本，如 Adam）更新参数。整个过程就是神经网络学习的核心。

为了简化表达，我们首先引入重要的中间变量。

Definition 1.3.6. (误差项，预激活值，激活值)

对于第 l 层的第 i 个神经元，定义其**预激活值** $z_i^{(l)}$ 为加权和加上偏置的结果，即

$$z_i^{(l)} = \sum_j W_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$$

其中 $a_j^{(l-1)}$ 是上一层的**激活值**（即上一层的输出）。经过激活函数 f 后得到本层的激活值 $a_i^{(l)} = f(z_i^{(l)})$。

定义该神经元的**误差项** $\delta_i^{(l)}$ 为损失函数 J 对预激活值 $z_i^{(l)}$ 的偏导数：

$$\delta_i^{(l)} = \frac{\partial J}{\partial z_i^{(l)}}.$$

误差项 $\delta_i^{(l)}$ 衡量了该神经元对最终损失的“责任”大小：如果 $\delta_i^{(l)}$ 很大，说明该神经元的预激活值对损失的影响很大，调整它就非常重要。

然后我们再来看反向传播的流程。

第一步，前向传播。给定一个输入样本 x，我们首先按照网络的前向顺序，逐层计算并保存每一层的预激活值 $z^{(l)}$ 和激活值 $a^{(l)}$（记 $a^{(0)} = x$）。这些值在后面反向传播中会反复用到。

接着，计算输出层的误差。首先回顾定义：误差项 $\delta_i^{(l)} = \frac{\partial J}{\partial z_i^{(l)}}$。对于输出层 $l = L$，我们要计算的就是 $\frac{\partial J}{\partial z_i^{(L)}}$。但是 J 并不是 $z_i^{(L)}$ 的直接函数——J 依赖于输出层的激活值 $a^{(L)}$，而 $a^{(L)}$ 又是通过激活函数 f 从 $z^{(L)}$ 得到的。所以我们需要用链式法则把这两层关系连接起来。具体来说，对于输出层的第 i 个神经元，有：

$$\delta_i^{(L)} = \frac{\partial J}{\partial z_i^{(L)}} = \frac{\partial J}{\partial a_i^{(L)}} \cdot \frac{\partial a_i^{(L)}}{\partial z_i^{(L)}}$$

这里 $\frac{\partial J}{\partial a_i^{(L)}}$ 是损失对第 i 个输出激活值的偏导，而依据激活函数的定义：激活值 $a_i^{(L)}$ 是通过将激活函数 f 应用到预激活值 $z_i^{(L)}$ 上得到的，即

$$a_i^{(L)} = f(z_i^{(L)})$$

这里，f 是一个确定的函数（比如 Sigmoid、tanh 或 ReLU，后文详细展开说明），它接受一个实数输入，输出一个实数。当我们对 $a_i^{(L)}$ 关于 $z_i^{(L)}$ 求偏导时，本质上就是在问：“如果 $z_i^{(L)}$ 发

生微小变化，$a_i^{(L)}$ 会变化多少？”这正是导数 $\frac{df(z)}{dz}$ 在 $z = z_i^{(L)}$ 处的值。因此：

$$\frac{\partial a_i^{(L)}}{\partial z_i^{(L)}} = f'(z_i^{(L)}) \implies \delta_i^{(L)} = \frac{\partial J}{\partial z_i^{(L)}} = \frac{\partial J}{\partial a_i^{(L)}} \cdot \frac{\partial a_i^{(L)}}{\partial z_i^{(L)}} = \frac{\partial J}{\partial a_i^{(L)}} f'(z_i^{(L)})$$

这里 $f'(z_i^{(L)})$ 就是激活函数 f 在 $z_i^{(L)}$ 这一点的导数。因此，我们如果把所有神经元写在一起，就得到了向量形式的公式：

$$\delta^{(L)} = \nabla_{a^{(L)}} J \odot f'(z^{(L)})$$

其中 $\nabla_{a^{(L)}} J$ 是由所有 $\frac{\partial J}{\partial a_i^{(L)}}$ 组成的向量，$\odot$ 表示逐元素相乘。

Definition 1.3.7. (*Hadamard* 积（逐元素相乘））

设 $\mathbf{u} = (u_1, u_2, \ldots, u_n)$ 和 $\mathbf{v} = (v_1, v_2, \ldots, v_n)$ 是两个维度相同的向量。它们的 **Hadamard 积**（也称为逐元素乘积）记作 $\mathbf{u} \odot \mathbf{v}$，定义为一个同样维度的向量，其第 i 个分量为对应分量的乘积：

$$(\mathbf{u} \odot \mathbf{v})_i = u_i \cdot v_i, \quad i = 1, 2, \ldots, n.$$

这一运算也可推广到矩阵：对于两个同型矩阵 $A, B \in \mathbb{R}^{m \times n}$，其 Hadamard 积 $(A \odot B)_{ij} = A_{ij} B_{ij}$。在反向传播中，Hadamard 积常用于对向量进行逐元素的调制。例如，输出层误差公式 $\delta^{(L)} = \nabla_{a^{(L)}} J \odot f'(z^{(L)})$ 的含义是：将损失对每个输出激活值的偏导与对应神经元的激活函数导数逐元素相乘，从而得到每个预激活值的偏导。

所以，输出层的误差由两部分组成——损失对输出的敏感度 $\nabla_{a^{(L)}} J$ 和激活函数本身的局部尺度 $f'(z^{(L)})$。接下来我们来看两个常见例子：

- 如果使用均方误差损失且输出层没有激活函数（线性输出），则 $\nabla_{a^{(L)}} J = a^{(L)} - y$，因此

$$\delta^{(L)} = (a^{(L)} - y) \odot 1 = a^{(L)} - y.$$

- 如果使用交叉熵损失且输出层采用 Softmax 激活，那么有一个著名的简化结果：$\nabla_{a^{(L)}} J = a^{(L)} - y$，所以 $\delta^{(L)}$ 同样等于 $a^{(L)} - y$。这个巧合大大简化了分类任务的实现。

有了输出层的误差，我们就可以利用链式法则，将误差向后一层一层地传播。我们首先从权重梯度的计算开始，然后自然地导出误差反向传播的递推关系。

1.3.4 计算权重梯度

考虑第 l 层的权重 $W_{ij}^{(l)}$，它连接了第 $l-1$ 层的第 j 个神经元（激活值为 $a_j^{(l-1)}$）和第 l 层的第 i 个神经元（预激活值为 $z_i^{(l)}$）。我们的目标是求损失 J 对该权重的偏导数 $\dfrac{\partial J}{\partial W_{ij}^{(l)}}$。注意，$J$ 是通过 $z_i^{(l)}$ 依赖于 $W_{ij}^{(l)}$ 的，因为 $z_i^{(l)} = \sum_k W_{ik}^{(l)} a_k^{(l-1)} + b_i^{(l)}$。根据链式法则：

$$\frac{\partial J}{\partial W_{ij}^{(l)}} = \frac{\partial J}{\partial z_i^{(l)}} \cdot \frac{\partial z_i^{(l)}}{\partial W_{ij}^{(l)}}$$

第一项正是我们定义的误差项 $\delta_i^{(l)} = \frac{\partial J}{\partial z_i^{(l)}}$。第二项是 $z_i^{(l)}$ 对 $W_{ij}^{(l)}$ 的偏导，从 $z_i^{(l)}$ 的表达式可以看出，只有 $k = j$ 的那一项有贡献，因此

$$\frac{\partial z_i^{(l)}}{\partial W_{ij}^{(l)}} = a_j^{(l-1)} \implies \frac{\partial J}{\partial W_{ij}^{(l)}} = \frac{\partial J}{\partial z_i^{(l)}} \frac{\partial z_i^{(l)}}{\partial W_{ij}^{(l)}} = \delta_i^{(l)} a_j^{(l-1)}$$

现在要把所有这些偏导数组织成一个矩阵 $\dfrac{\partial J}{\partial W^{(l)}}$，这个矩阵的第 i 行第 j 列元素就是 $\delta_i^{(l)} a_j^{(l-1)}$。

为了用向量运算简洁地表达这个矩阵，我们注意到 $\delta^{(l)}$ 是一个列向量，长度为第 l 层的神经元个数；$a^{(l-1)}$ 也是一个列向量，长度为第 $l-1$ 层的神经元个数。它们的外积 $\delta^{(l)}(a^{(l-1)})^\mathsf{T}$ 定义为：第一个向量（列）乘以第二个向量的转置（行），结果是一个矩阵，其第 (i,j) 元素正好是 $\delta_i^{(l)}$ 乘以 $a_j^{(l-1)}$。这正是我们需要的。

$$\frac{\partial J}{\partial W^{(l)}} = \delta^{(l)}(a^{(l-1)})^\mathsf{T}$$

这里的转置符号 $(a^{(l-1)})^\mathsf{T}$ 将列向量变为行向量，从而通过矩阵乘法得到期望的矩阵。类似地，对于偏置 $b_i^{(l)}$，有 $z_i^{(l)} = \sum_k W_{ik}^{(l)} a_k^{(l-1)} + b_i^{(l)}$，所以 $\dfrac{\partial z_i^{(l)}}{\partial b_i^{(l)}} = 1$，因此

$$\frac{\partial J}{\partial b_i^{(l)}} = \delta_i^{(l)}, \quad \text{即} \frac{\partial J}{\partial b^{(l)}} = \delta^{(l)}$$

现在我们需要知道如何计算非输出层的误差项 $\delta^{(l)}$（$l < L$），因为上面权重梯度的表达式依赖于它。考虑第 l 层的误差项 $\delta_i^{(l)} = \frac{\partial J}{\partial z_i^{(l)}}$。$J$ 对 $z_i^{(l)}$ 的依赖是通过下一层（第 $l+1$ 层）的所有预激活值 $z_j^{(l+1)}$ 实现的，因为 $z_j^{(l+1)} = \sum_k W_{jk}^{(l+1)} a_k^{(l)} + b_j^{(l+1)}$，而 $a_k^{(l)} = f(z_k^{(l)})$。应用链式法则：

$$\delta_i^{(l)} = \sum_j \frac{\partial J}{\partial z_j^{(l+1)}} \cdot \frac{\partial z_j^{(l+1)}}{\partial z_i^{(l)}} = \sum_j \delta_j^{(l+1)} \cdot \frac{\partial z_j^{(l+1)}}{\partial z_i^{(l)}}.$$

现在计算 $\frac{\partial z_j^{(l+1)}}{\partial z_i^{(l)}}$。从 $z_j^{(l+1)}$ 的表达式，$z_j^{(l+1)} = \sum_k W_{jk}^{(l+1)} f(z_k^{(l)}) + b_j^{(l+1)}$，对 $z_i^{(l)}$ 求偏导时，只有 $k = i$ 的那一项有贡献，且 $f(z_i^{(l)})$ 的导数为 $f'(z_i^{(l)})$，所以

$$\frac{\partial z_j^{(l+1)}}{\partial z_i^{(l)}} = W_{ji}^{(l+1)} f'(z_i^{(l)}).$$

代入上式：

$$\delta_i^{(l)} = \sum_j \delta_j^{(l+1)} W_{ji}^{(l+1)} f'(z_i^{(l)}) = \left(\sum_j (W^{(l+1)})_{ji} \delta_j^{(l+1)} \right) f'(z_i^{(l)}).$$

注意求和 $\sum_j (W^{(l+1)})_{ji} \delta_j^{(l+1)}$ 正是矩阵 $(W^{(l+1)})^\mathsf{T}$ 与向量 $\delta^{(l+1)}$ 乘积的第 i 个分量。因此，对所有 i 写为向量形式，我们得到反向传播的核心递推公式：

$$\boxed{\delta^{(l)} = \left((W^{(l+1)})^\mathsf{T} \delta^{(l+1)} \right) \odot f'(z^{(l)})}.$$

至此，我们完成了所有公式的推导。整个过程可以总结为：

- 从输出层开始，先计算 $\delta^{(L)}$（根据损失函数和激活函数）；
- 然后利用递推公式，从后向前逐层计算 $\delta^{(l)}$；
- 最后用 $\delta^{(l)}$ 和上一层的激活值 $a^{(l-1)}$ 计算权重梯度 $\frac{\partial J}{\partial W^{(l)}}$ 和偏置梯度 $\frac{\partial J}{\partial b^{(l)}}$。

这些公式在代码中可以直接用矩阵运算实现，非常高效。它们的直观意义也很清晰：后一层的误差通过权重矩阵的转置“分配”回前一层，再乘以前一层激活函数的导数进行调制，从而得到前一层的误差。这正是“反向传播”名称的由来——误差信号从输出层反向流动到输入层。从直观上看，后一层的误差 $\delta^{(l+1)}$ 首先通过权重矩阵的转置 $(W^{(l+1)})^\mathsf{T}$“分配”回前一层——这相当于把错误按连接权重的大小分摊给每个前层神经元。然后乘以本层激活函数

的导数 $f'(z^{(l)})$，用来修正该神经元当时的“敏感度”。如果某神经元当时处于饱和区（导数接近 0），那么即使分摊到的错误很大，它也不会认为这是自己的责任，因为它的输出几乎不受输入影响。

一旦得到了 $\delta^{(l+1)}$，第 l 层参数的梯度就可以用非常简洁的形式表达：

$$\frac{\partial J}{\partial W^{(l)}} = \delta^{(l+1)}(a^{(l)})^{\mathsf{T}}, \qquad \frac{\partial J}{\partial b^{(l)}} = \delta^{(l+1)}$$

注意这里 $\delta^{(l+1)}$ 是一个列向量，$(a^{(l)})^{\mathsf{T}}$ 是行向量，它们的乘积（外积）得到一个矩阵，正是权重梯度的每个元素。偏置的梯度就是误差本身，这也很直观：误差项已经衡量了预激活值对损失的影响，而偏置直接加在预激活值上，所以它的梯度自然就是 δ。

有了梯度之后，我们就可以用梯度下降（或其变体）更新参数：

$$W^{(l)} \leftarrow W^{(l)} - \alpha\frac{\partial J}{\partial W^{(l)}}, \qquad b^{(l)} \leftarrow b^{(l)} - \alpha\frac{\partial J}{\partial b^{(l)}}.$$

在实际训练中，我们不会只用一个样本计算梯度（那样波动太大，且计算效率低），而是采用**小批量**（mini-batch）方式：每次选取一小批样本，分别计算每个样本的梯度，然后取平均作为该批次的最终梯度。这样做既利用了向量化加速，又引入了适当的噪声，有助于逃离局部极小点。

反向传播与梯度下降的结合，构成了神经网络训练的完整框架：

- 前向传播计算预测值和中间变量；
- 反向传播计算所有参数的梯度；
- 梯度下降利用梯度更新参数；
- 不断重复直到损失收敛。

这个过程在深度学习中被称为“训练”或“学习”。理解它，你就掌握了神经网络的“引擎”。接下来，我们将在这个基础上讨论各种激活函数、优化技巧以及网络结构的设计。

Sec 1.4 常见的激活函数及其特点

在前一节中，我们详细推导了神经网络训练的完整流程：前向传播计算预测值，反向传播计算梯度，梯度下降更新参数。在这个过程中，激活函数 f 扮演着至关重要的角色——它引入非线性，并且它的导数 $f'(z)$ 直接出现在反向传播的误差递推公式 $\delta^{(l)} = ((W^{(l+1)})^{\mathsf{T}}\delta^{(l+1)}) \odot f'(z^{(l)})$ 中。因此，激活函数的选择直接影响着梯度能否有效地在深层网络中流动。

在深入具体的激活函数之前，我们先思考一个问题：什么样的函数适合作为神经元的激活函数？从数学和工程角度，理想的激活函数通常应具备以下性质：

- **非线性**：这是激活函数的根本。如果激活函数是线性的，那么无论堆叠多少层，整个网络仍然等价于一个线性模型，无法学习复杂的模式。非线性赋予了网络“表达”复杂函数的能力。
- **可微性（或至少分段可微）**：神经网络的训练依赖于反向传播计算梯度，这要求激活函数在大多数点上有定义良好的导数（允许个别不可导点，如 ReLU 在 0 处，实际训练中不影响）。

- **单调性**：单调的激活函数（即随着输入增大，输出也单调增大或减小）有助于损失函数在参数空间保持较好的形状，使优化过程更加稳定。
- **有界性（非强制，但很有用）**：有界输出可以限制激活值的范围，防止数值爆炸。同时，如果输出范围在 $[0,1]$ 或 $[-1,1]$ 之间，可以方便地解释为概率或某种归一化的特征。
- **计算简单**：在深度网络中，激活函数被调用的次数极其频繁。简单的运算（如比较大小、加减乘除）比复杂的运算（如指数、对数）快得多，能显著节省训练和推理时间。
- **导数的尺度适中**：从反向传播的递推公式可以看出，误差信号每经过一层就要乘上该层激活函数的导数。如果导数太小，梯度会指数级衰减（梯度消失）；如果导数太大，梯度可能指数级爆炸（梯度爆炸）。因此，我们希望导数在大部分区域都不太小也不太大。

历史上，研究者们正是围绕这些性质不断探索，才有了从 Sigmoid 到 ReLU 的演进。下面我们逐一剖析几种经典激活函数，看看它们如何满足（或违背）这些要求，并因此被广泛采用或逐渐取代。

Definition 1.4.8. (*Sigmoid* 函数：概率解释的经典选择)

Sigmoid 函数是神经网络早期最常用的激活函数之一，其定义为：

$$\sigma(z) = \frac{1}{1+e^{-z}} \in (0,1)$$

从几何上看，它将整个实数轴平滑地压缩到 $(0,1)$ 区间。当 z 很大时，$\sigma(z)$ 趋近于 1；当 z 很小时，趋近于 0；在 $z=0$ 附近，它近似线性。这个特性使得 Sigmoid 的输出天然适合解释为“激活概率”——例如，在二分类问题中，我们可以把输出理解为样本属于正类的概率。主要作为二分类问题的输出层激活函数，将线性输出转换为概率。在隐层中，Sigmoid 已基本被更优的激活函数取代。人们将其作为激活函数的理由：

1. **非线性**：Sigmoid 是光滑的非线性函数，满足激活函数的根本要求。
2. **可微且导数简洁**：$\sigma'(z) = \sigma(z)(1-\sigma(z))$，已知输出即可得导数，计算方便。
3. **有界且单调**：输出范围 $(0,1)$ 单调递增，有利于稳定训练。
4. **概率解释**：值域恰好对应概率空间，使其在输出层成为自然选择。

但是，这个函数也有缺点，Sigmoid 的输出范围是 $(0,1)$，始终大于 0，平均值约为 0.5。这意味着下一层神经元的输入全部是正数。这会给梯度下降带来一个麻烦：在更新权重时，每个权重的梯度方向会被限制——从权重更新公式

$$\frac{\partial J}{\partial W^{(l)}} = \delta^{(l+1)}(a^{(l)})^{\mathsf{T}}$$

可以看出，梯度的符号完全由 $\delta^{(l+1)}$ 的符号决定，而 $a^{(l)}$ 全为正，因此所有权重的梯度符号都相同（要么全正，要么全负）。这导致权重更新路径像“锯齿”一样来回震荡，无法直接沿着最优方向下降，从而减慢了收敛速度。

Definition 1.4.9. (双曲正切函数)

为了缓解 Sigmoid 非零中心的问题，tanh 函数被引入：

$$\tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}} \in (-1,1)$$

它在形状上与 Sigmoid 相似，都是平滑的 S 形曲线，但输出范围变为 $(-1,1)$，均值为 0。

由于输出均值接近 0，下一层的输入不会产生系统性偏移，从而避免了 Sigmoid 的“锯齿”问题，梯度下降的效率更高，收敛速度通常快于 Sigmoid，而且同样具备非线性、可微、单调、有界等优点，且导数形式也很简洁：$\tanh'(z) = 1 - \tanh^2(z)$。

但是这两个激活函数都有相同的缺点，首先，公式中都包含指数运算 e^{-z}，而指数运算在计算机中比简单的加减法和比较操作要慢得多。在深度网络中，激活函数会被调用亿万次，即使是微小的计算开销也会累积成巨大的训练成本。

更重要的是，如果我们回忆反向传播的推导，误差信号从输出层向输入层传播时，每经过一层就要乘以该层激活函数的导数 $f'(z)$。对于 Sigmoid 函数，当 $|z|$ 较大时（例如 $z > 5$ 或 $z < -5$），函数进入“饱和区”——曲线变得非常平坦，导数 $\sigma'(z)$ 趋近于 0。在深层网络中，如果许多神经元都处于饱和区，误差信号经过它们时就会反复乘上很小的数。假设网络有 L 层，每层导数平均为 0.1，那么传到第一层时信号就变成了原来的 0.1^L，几乎为零。这意味着浅层神经元接收不到有效的更新指令，学习极其缓慢甚至停滞。这正是早期深度网络难以训练的主要原因，称为**梯度消失**（Vanishing Gradient）。同理，tanh 也并未从根本上解决梯度消失问题。

梯度消失问题长期困扰着神经网络的发展，直到 ReLU 的出现带来了根本性的改变。

Definition 1.4.10. (线性整流单元 *ReLU*)

线性整流单元（Rectified Linear Unit, ReLU）的定义极其简单：

$$\mathrm{ReLU}(z) = \max(0, z)$$

它在正半轴是恒等映射，在负半轴恒为 0。

这个函数作为激活函数的优势如下：

1. **计算极其高效**：只需一次比较操作，没有任何指数或除法运算，大大加快了前向和反向传播的速度。
2. **有效缓解梯度消失**：在正半轴，ReLU 的导数恒为 1。这意味着误差信号在通过处于正半轴的神经元时可以无损地反向传播，不会像 Sigmoid/tanh 那样被衰减。即使网络很深，浅层神经元也能接收到足够强度的梯度信号进行更新。这是 ReLU 成为深度网络主流选择的最关键原因。
3. **稀疏激活**：在负半轴，ReLU 输出 0，相当于让部分神经元“静默”。这种稀疏性有助于网络去冗余，使特征表示更具选择性。从生物学视角看，这类似于真实神经元“抑制”或“不响应”的状态。
4. **收敛速度显著提升**：得益于梯度的良好流动和稀疏性，使用 ReLU 的网络通常比 Sigmoid/tanh 网络收敛快得多。

然而，正是这种“简单粗暴”的特性，也带来了新的挑战。ReLU 的设计思路是在正区间完全保留信号，负区间完全阻断信号——这种非黑即白的处理方式在某些情况下会适得其反。具体来说，ReLU 在实际应用中存在以下缺陷：

1. **死亡 ReLU 问题（Dying ReLU）**：这是 ReLU 最著名的隐患。当某神经元的输入总是落在负半轴（例如，由于初始化不当、学习率过大导致权重更新过猛），其输出恒为 0，导数也恒为 0，梯度无法流经此神经元。一旦陷入这种状态，该神经元将永久“死亡”，再也无法被激活。在实际训练中，如果学习率设置过高，可能相当比例的神经元会死亡，从而削弱模型容量。针对这一问题，研究者提出了各种改进变体，如 Leaky ReLU（在负

半轴保留一个小斜率）、ELU（指数线性单元）等，它们在负半轴仍允许少量梯度通过。

2. **输出非零中心**：ReLU 的输出总是非负，仍存在 Sigmoid 式的偏移问题，但相比于梯度消失的改善，这一缺点通常可以接受。
3. **无上界**：ReLU 的输出没有上限，如果某些神经元的输出变得非常大，那么在反向传播中，梯度也可能变得非常大，导致参数更新过猛，损失函数震荡甚至发散，这种现象称为**梯度爆炸**（Gradient Explosion）。不过，这一问题可以通过适当的参数初始化、较小的学习率，或者在梯度更新前将梯度的模长截断到一个合理范围——即**梯度裁剪**（Gradient Clipping）——来有效控制。

针对上述问题，研究者们提出了多种 ReLU 的改进变体，试图在保留 ReLU 计算高效、梯度流畅等优点的同时，弥补其缺陷。在介绍这些变体之前，我们需要先明确一个概念：这些变体中往往包含一些需要人为设定的参数——例如负半轴的斜率、指数衰减的幅度等——它们控制着激活函数的形状，却不通过训练数据自动学习（除非特别设计）。这类参数在机器学习中被称为超参数。

Definition 1.4.11. (超参数（*Hyperparameter*）)

在机器学习中，超参数是指那些在训练开始前需要人为设定的参数，它们不通过训练数据自动学习得到，而是控制模型的训练过程和结构。与模型参数（如神经网络的权重和偏置）不同，超参数的值无法通过梯度下降等优化算法更新，通常需要根据经验、领域知识或通过搜索方法（如网格搜索、随机搜索）来确定。常见超参数包括：

- **学习率** η：控制参数更新的步长。
- **批次大小**（batch size）：每次训练使用的样本数。
- **迭代轮数**（epochs）：整个数据集被遍历的次数。
- **网络结构超参数**：隐藏层数量、每层神经元个数、激活函数类型等。
- **正则化参数**：如 L1/L2 正则化系数、Dropout 概率。
- **优化器参数**：如动量系数、Adam 的 β_1, β_2 等。

选择合适的超参数对模型性能至关重要，通常需要通过实验调优。

理解了超参数的概念，我们再来看 ReLU 的改进变体，就会发现它们正是在“如何设计这些可调参数”这一问题上做出了不同的探索：有的将参数固定为经验值（如 Leaky ReLU），有的将参数也纳入学习过程（如 PReLU），有的则设计了更复杂的函数形式（如 ELU、GELU）。下面介绍几种最具代表性的变体。

- **Leaky ReLU（带泄漏的线性整流单元）**：定义为 $f(z) = \max(\alpha z, z)$，其中 α 是一个小的正常数（通常取 0.01）。它在负半轴不再恒为 0，而是保留了一个微小的斜率，使得负值输入也能产生非零输出和梯度。这从根本上避免了神经元死亡的问题——即使输入长期为负，神经元仍能接收到微弱的更新信号，有机会“复活”。Leaky ReLU 几乎不增加计算成本，且在很多任务上表现优于原始 ReLU。其缺点是 α 需要手动设定，且取值对性能有一定影响。
- **PReLU（Parametric ReLU，参数化线性整流单元）**：PReLU 将 Leaky ReLU 中的负半轴斜率 α 变为可训练的参数，让网络在训练过程中自行学习每个通道（或每个神经元）最适合的斜率值。数学形式与 Leaky ReLU 相同，但 α 不再固定。这种方法增加了少量参数，但赋予了网络更大的灵活性，在图像分类等任务上取得了比 ReLU 更好的效果。需

要注意的是，PReLU 在负半轴仍是线性函数，因此计算依然高效。

- **ELU（Exponential Linear Unit，指数线性单元）**：定义为

$$f(z) = \begin{cases} z, & z > 0, \\ \alpha(e^z - 1), & z \le 0, \end{cases}$$

其中 $\alpha > 0$ 是一个超参数（通常取 1）。ELU 在正半轴保持 ReLU 的线性，在负半轴则采用指数衰减的形式，使输出平滑地趋近于 $-\alpha$。这一设计有两个优点：一是负半轴的输出均值更接近 0，有助于缓解偏移问题；二是导数在 $z = 0$ 处连续（左导数为 α，右导数为 1，需设置 $\alpha = 1$ 才连续），且负半轴的梯度不会像 Leaky ReLU 那样突然跳变，可能使优化更稳定。缺点是计算中涉及指数运算，比 ReLU 稍慢。

- **GELU（Gaussian Error Linear Unit，高斯误差线性单元）与 SiLU（Sigmoid Linear Unit）**：GELU 定义为 $f(z) = z \cdot \Phi(z)$，其中 $\Phi(z)$ 是标准正态分布的累积分布函数；SiLU（也被称为 Swish）定义为 $f(z) = z \cdot \sigma(z)$，其中 $\sigma(z)$ 是 Sigmoid 函数。两者形式相似，都是将输入乘以其自身的“门控”值，形成一种平滑的非单调激活函数。这类函数在负半轴不是简单的线性或指数形式，而是先略为负值再回升，产生一个微小的负向突起。这种非单调性被证明有助于模型学习更复杂的特征。GELU 和 SiLU 在 BERT、GPT 等 Transformer 架构中成为默认选择，表现出优异的性能。它们的缺点是计算相对复杂（涉及 Sigmoid 或正态分布函数），且缺乏直观的几何解释。

表 1.1: 激活函数对比

函数	输出范围	零中心	梯度消失	计算速度
Sigmoid	(0,1)	否	严重	慢
tanh	(-1,1)	**是**	严重	慢
ReLU	$[0, \infty)$	否	**缓解**	**极快**

1.4.1 发行版安装与更新

本模板测试环境为

1. Win11 23H2 + TEX Live 2024；

TEXLive/MacTEX 的安装请参考知乎的文章, 此处略过。

安装 TEX Live 之后，安装后建议升级全部宏包，升级方法：使用 cmd 或 terminal 运行 `tlmgr update --all`，如果 tlmgr 需要更新，请使用 cmd 运行 `tlmgr update --self`，如果更新过程中出现了中断，请改用 `tlmgr update --self --all --reinstall-forcibly-removed` 更新，也即

```
tlmgr update --self
tlmgr update --all
tlmgr update --self --all --reinstall-forcibly-removed
```

更多的内容请参考 How do I update my TEX distribution?

1.4.2 其他发行版本

由于宏包版本问题，本模板不支持 CTEX 套装，请务必安装 TeX Live/MacTEX。更多关于 TEX Live 的安装使用以及 CTEX 与 TEX Live 的兼容、系统路径问题，请参考官方文档。

BEAUTYBOOK 设置说明

Part I

本模板英文版基于基础的 book 文类, 中文版则基于 ctexbook 文类，所以 book 或者 ctexbook 的选项对于本模板也是有效的。默认编码为 UTF-8，推荐使用 TEX Live 编译。

Sec 2.1 语言模式

本模板内含两套基础语言环境, 分别为中文的`beautybook-CN.cls`、英文的`beautybook-EN.cls`。改变语言环境会改变图表标题的引导词（图，表），文章结构词（比如目录，参考文献等），以及定理环境中的引导词（比如定理，引理等）。不同语言模式的启用如下：

```
\documentclass{beautybook-CN} % 中文
\documentclass{beautybook-EN} % 英文
```

除模板自带的两套语言设定之外，如果您需要使用其他语言, 可以通过更改 cls 文件中这几处解决, 分别为

1. 更改 part 环境的名称 `Part \thepart`为（你的语言中`part`的翻译）`\thepart`
2. 主文件, 即当前文件导言区中的定理引导词
3. 更改 chapter 环境中的 part 名称如第一条所示
4. 记住, 仅有亚洲语言环境可以使用 ctexbook 文类, 即基于`beautybook-CN.cls`更改, 其他西语环境需要基于`beautybook-EN.cls`更改.

Sec 2.2 颜色主题

本模板的颜色是可以自由配置的，可以配置的颜色参数如下：

```
  \definecolor{bg}{HTML}{e0e0e0} % 整体风格的背景色 % 即浅色
  \definecolor{fg}{HTML}{455a64} % 整体风格的前景色  % 即深色
  %% 下面颜色位于 stys/bottompage.sty文件中
  \definecolor{coverbgcolor}{HTML}{f9b868} % 封面及封底背景色
  \definecolor{coverfgcolor}{HTML}{503D4B} % 封面及封底前景色
  \definecolor{coverbar}{HTML}{BF8E6F} % 封面竖条颜色
  \definecolor{bottomcolor}{HTML}{B3686A} % 封底说明背景颜色
  %%%%%%%%%%%%%%%%%%%%%%%%
  \colorlet{framegolden}{fg} % 古风盒子线条颜色
  \colorlet{framegray}{黛绿!5} % 古风盒子背景色
```

还有定理环境颜色可以在此文件的导言区设置, 下面数学环境部分会展开讲.

这里推荐使用林莲枝开发的 cncolours 宏包的颜色配置, 可以对照选取适合的颜色.

Sec 2.3 封面

2.3.1 封面个性化

本模板拥有多套封面可随意取用, 其中使用方法如下:

1. Springer 经典封面–对应宏包 `cover-choose=cn` (中文默认),
2. Springer 经典封面之二–对应宏包 `cover-choose=en` (英文默认),
3. Springer 经典封面之三–对应宏包 `cover-choose=enfig` (图片背景),
4. 中文书籍经典封面–对应宏包`cover-choose=birkar` (三角几何风)。注意, 使用该封面所对应的信息不太一样, 看好上面的示例，按照要求操作即可。

表 2.1: 封面元素信息

信息	命令	信息	命令	信息	命令
标题	`\title`	副标题	`\subtitle`	作者	`\author`
出版社	`\pressname`	版本	`\edition`	封面图	`\coverimage`
徽标	`\presslogo`				

2.3.2 封面图

封面图片可以自行选取.

2.3.3 徽标

本文用到的 Logo 为 wiki 随意找的 springer 经典马标, 可以自己查询下载出版社 logo, 为免侵权, 在更换图片的时候请选择合适合法的图片进行替换。

2.3.4 自定义封面

另外，如果使用自定义的封面，比如 Adobe illustrator 或者其他软件制作的 A4 PDF 文档，请把 `\makecover` 注释掉，然后借助 `pdfpages` 宏包将自制封面插入即可。如果使用 `titlepage` 环境，也是类似。

Sec 2.4 章标题

本模板自定义了一套标题样式, 主要是 part、chapter、section 这三个标题，具体代码见 cls。可能不适合所有人的审美，可以注释掉就会回归默认 ctexbook 的标题样式。

Sec 2.5 数学环境简介

在我们这个模板中，我们定义了四种不同的定理模式，包括简单模式 (默认的定理样式 amsthm) 、有点自定义的 thmtools、彩色强调盒子、以及本人开发的专有版权盒子，当然，由雾月老师给我定制的古风盒子您也可以是用来作为定理盒子，只需要在本文件导言区第一种定理样式里面加上`ys style`即可.

2.5.1 定理类环境的使用

以下是使用效果展示

2.5.1.1 amsthm

Remark. 这是基于 *amsthm* 的注释环境

2.5.1.2 thmtools

Proof **(证明的说明).** 证明环境 ■

Solution **(解的说明).** 解环境 ■

2.5.1.3 彩色强调盒子

Definition 2.5.1 (名称)**.** 第一种定义环境

Theorem 2.5.1 (名称)**.** 第一种定理环境

Corollary 2.1 (名称)**.** 第一种推论环境

Proposition 2.5.1 (名称)**.** 第一种命题环境

Example 2.1 (名称)**.** 第一种例题环境

Lemma 2.5.1 (名称)**.** 第一种引理环境

2.5.1.4 个人版权的盒子共两种

Definition 2.5.1. (名称)

这是采用个人定制的盒子制作的定理环境，这是其中定义环境示例。注意：使用方法如下

- 如果你没有名称和标签，使用方法为

```
\begin{definition}
    定义环境内容
\end{definition}
```

- 如果你没有标签但有名称，使用方法为

```
\begin{definition}[][名称]
    定义环境内容
\end{definition}
```

- 如果你有标签，那么无论是否有名称，使用方法为

```
\begin{definition}[][有就填，没有空着][标签]
    定义环境内容
\end{definition}
```

- 如果你想更改盒子的一些设定选项，比如加框线等之类的，使用方法为

```
\begin{definition}[tcolorbox选项][名称有就写，没有就连带外面括号删掉][标签（有标签下就这样子，没有标签可以把这个标签连带外面的括号删掉)]
    定义环境内容
\end{definition}
```

Theorem 2.5.1.

用法同上，引用下上面的标签 *2.5.1*或者可以***Definition 2.5.1***.

Lemma 2.5.1.

用法同上，引用下上面的标签 *2.5.1*或者可以***Definition 2.5.1***.

Corollary 2.5.1.

用法同上，引用下上面的标签 *2.5.1*或者可以***Definition 2.5.1***.

Example 2.5.1.

用法同上，引用下上面的标签 2.5.1或者可以**Definition 2.5.1**.

古风盒子

古风盒子测试，可以任意嵌套其他环境！

2.5.2 修改计数器

当前定理等环境计数器按章计数，如果想修改定理类环境按节计数，可以修改计数器选项 `counter/.code`中的`chapter`，可用选项为 `chapter` （默认）与 `section`、`subsection`等

2.5.3 自定义定理类环境

用户可以采用四种方式定义自己的定理环境，分别为 amsthm 与 thmtools, 这两种看宏包说明文档即可; 后面两种定理的定义方式为如本文件导言区:

```
  % 这是第一种
  \mynewtheorem{
      defi={\textbf{定义}}[section]{interior style={left color=ReD!8,right color=ReD!5!CyaN!50}, borderline west={1.5mm}{0mm}{ReD}}, % 类似模仿即可
  }

  % 下面是第二种
  \mynewtcbtheorem{
  % 这个 theorem 是环境名
  theorem={ % 第一种 :  圣诞礼盒风格
      counter=tcbthm,
      the counter=\thesection.\arabic{tcbthm},
      name=定理, % 它保存到 \theorem@name 里
      thmcolor=purple5,
      autoref name=\bfseries 定理,
      style={
      arc=3pt,breakable,enhanced,interior style={top color=purple5!5 ,middle color=purple5!1!nuanbai, bottom color=nuanbai},boxrule=0pt,top=8mm,
      fuzzy shadow={-0.6mm}{0.6mm}{0mm}{0.3mm}{white!50!gray},% 上
      fuzzy shadow={0.6mm}{-0.6mm}{0mm}{0.3mm}{fill=white!40!gray},%下
      opacityframe=0, opacityback=0.98,
      fontupper=\itshape, step={tcbthm},
```

```
        before pre=\smallskip, after app=\smallskip,
        overlay unbroken=\my@theorem@overlay@unbroken{\theorem@name\ \thetcbthm}{\theorem@thmcolor},
        overlay first=\my@theorem@overlay@first{\theorem@name\ \thetcbthm}{\theorem@thmcolor},
        overlay last=\my@theorem@overlay@last{\theorem@thmcolor},
        }
    },
    lemma={ % 第二种 :  丝带风格
        counter=tcblem,
        the counter=\thesection.\arabic{tcblem},
        name=引理,
        lemcolor=靛蓝,
        autoref name=\bfseries 引理,
        style={
        arc=0mm,breakable,enhanced,interior style={top color=靛蓝!5 ,middle color=靛蓝!1!nuanbai, bottom color=nuanbai},arc=3pt,boxrule=0pt,top=7mm,bottom=5mm,
        fuzzy shadow={-0.6mm}{0.6mm}{0mm}{0.3mm}{white!50!gray},% 上
        fuzzy shadow={0.6mm}{-0.6mm}{0mm}{0.3mm}{fill=white!40!gray},%下
        opacityframe=0, opacityback=0.98,
        fontupper=\normalsize,step={tcblem},
        before pre=\smallskip, after app=\smallskip,
        overlay unbroken=\my@lemma@overlay@unbroken{\lemma@name\ \thetcblem}{\lemma@lemcolor},
        overlay first=\my@lemma@overlay@first{\lemma@name\ \thetcblem}{\lemma@lemcolor},
        overlay last=\my@lemma@overlay@last{\lemma@lemcolor},
        }
    },
}
```

Remark. 解释一下，其中的 *overlay* 部分更改需要看中文修改，定理名称改成你想要的，颜色也是，然后别忘了给最外面的 *example* 之类的环境名改成你的，比如 *axiom* 之类，还有就是 *tcbexam* 这个计数器名称要换成你新定义的，如 *tcbaxiom* 之类，其他就不用动了。至于说第一种定理样式看上面例子相信您能学会的。

Sec 2.6 列表环境

本模板借助于 `enumitem` 实现了可定制化，具体见 enumitem 宏包说明文档，这里示例如下

- first item of nesti;
- second item of nesti;
  - first item of nestii;
  - second item of nestii;
    - first item of nestiii;
    - second item of nestiii.

1) first item of nesti;
2) second item of nesti;
   (a) first item of nestii;
   (b) second item of nestii;
      i. first item of nestiii;
      ii. second item of nestiii.

Sec 2.7 参考文献

2.7.1 打印文献

`ref.bib` 为参考文献存放的文件，需要放在项目文件夹下。

2.7.2 修改文献格式

此外，本模板调用了 biblatex 宏包，并提供了 biber 引擎编译参考文献，当然您也可以直接删除 cls 中的 biblatex 宏包 (cls 最后几行) 来使用 bibtex.

关于文献条目（bib item），你可以在谷歌学术，Mendeley，Endnote 中取，然后把它们添加到 `ref.bib` 中。在文中引用的时候，引用它们的键值（bib key）即可。

文献样式默认为数字样式。

```
\usepackage[
backend=biber, % 可改为bibtex
style=numeric, % 可改为其他样式，参考biblatex说明文档
sorting=nty
]{biblatex}
\addbibresource{ref.bib}
```

3 字体选项

Part I

字体选项独立成章的原因是，我们希望本模板的用户关心模板使用的字体，知晓自己使用的字体以及遇到字体相关的问题能更加便捷地找到答案。

本模板默认使用 ctex 的 windows 选项提供的字体, 如非必要，字体不应改动，当然，如果确实需要，可按照下面代码操作：

```
 \setCJKmainfont[Path=fonts/,BoldFont={XX.TTF},ItalicFont={YY.TTF},SlantedFont = {ZZ.TTF} , SlantedFeatures = {FakeSlant}]{WW.TTF}
 \setCJKsansfont[Path=fonts/,BoldFont={XX.TTF},ItalicFont={XX.TTF}]{XX.TTF}
 \setCJKmonofont[Path=fonts/,BoldFont={XX.TTF},ItalicFont={XX.TTF}]{XX.TTF}
 %设置新的中文字体命令
 \newCJKfontfamily[song]\songti{XX.TTF}[Path=fonts/] %宋体
 %设置新的英文字体命令
 \newfontfamily\largetitlestyle[Path=fonts/]{XX.TTF}
```

！必须全部使用英文字体名称进行导入，否则报错找不到的!! 切记!!

Sec 3.1 数学字体选项

本模板使用的是默认数学字体，仅将 ∂ 符号改为 Times New Roman 的 Unicode 符号。

封底信息.

ISBN 978-80-7340-097-2

9 788073 400972

Beautybook